

Dynamic Random-Access Memory (DRAM)

■ Key features

- **DRAM** is traditionally packaged as a chip
- Basic storage unit is normally a **cell** (one bit per cell)
- Multiple DRAM chips form main memory in most computers

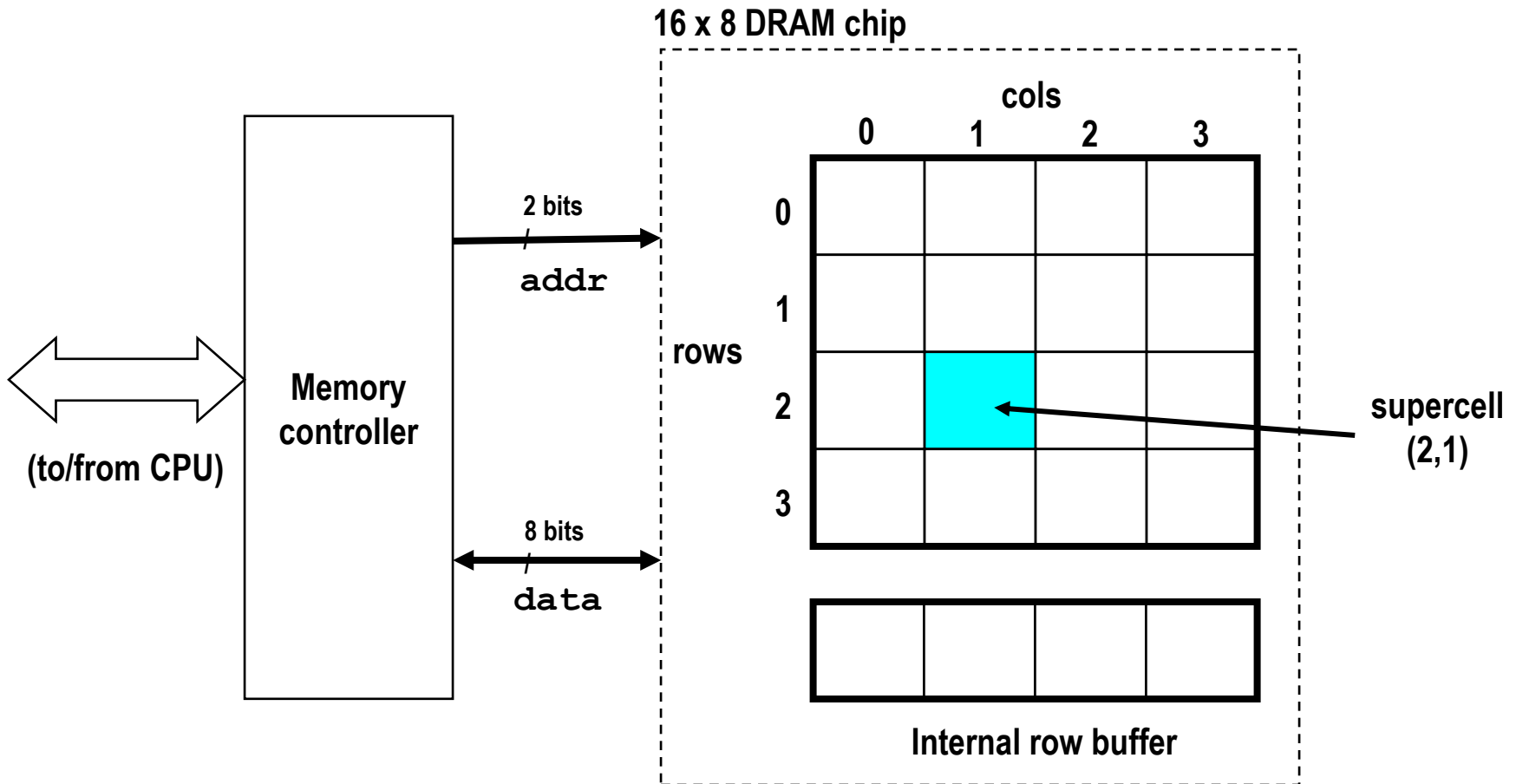
■ Technical characteristics

- Organized in two dimensions (rows and columns)
 - To access (within a DRAM chip): select row then select column
 - Consequence: 2nd access to a row faster than different column/row
- Each cell stores bit with a capacitor; one transistor is used for access
- Value must be refreshed every 10-100 ms
 - Done within the hardware

Conventional DRAM Organization

■ d x w DRAM:

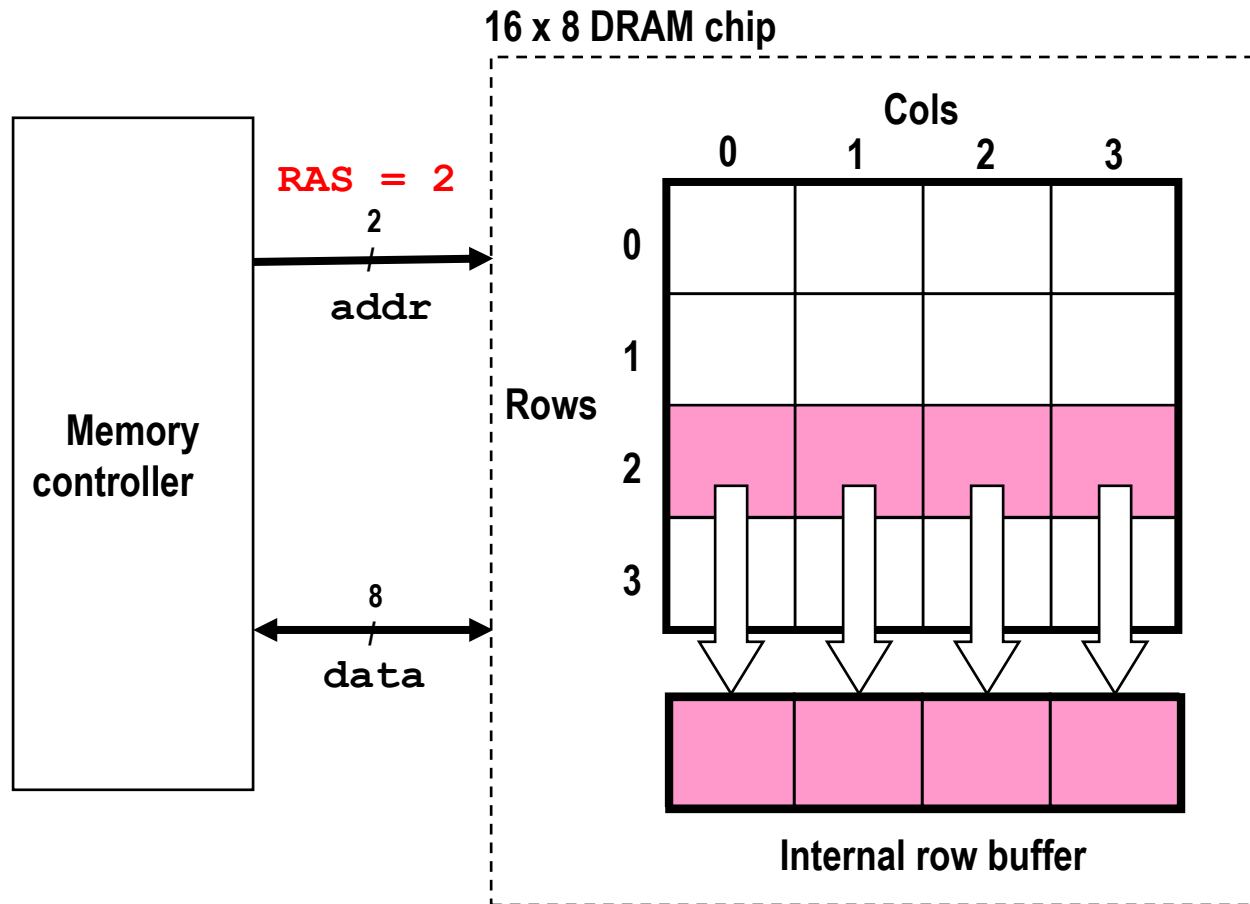
- dw total bits organized as d **supercells** of size w bits



Reading DRAM Supercell (2,1)

Step 1(a): Row access strobe (**RAS**) selects row 2.

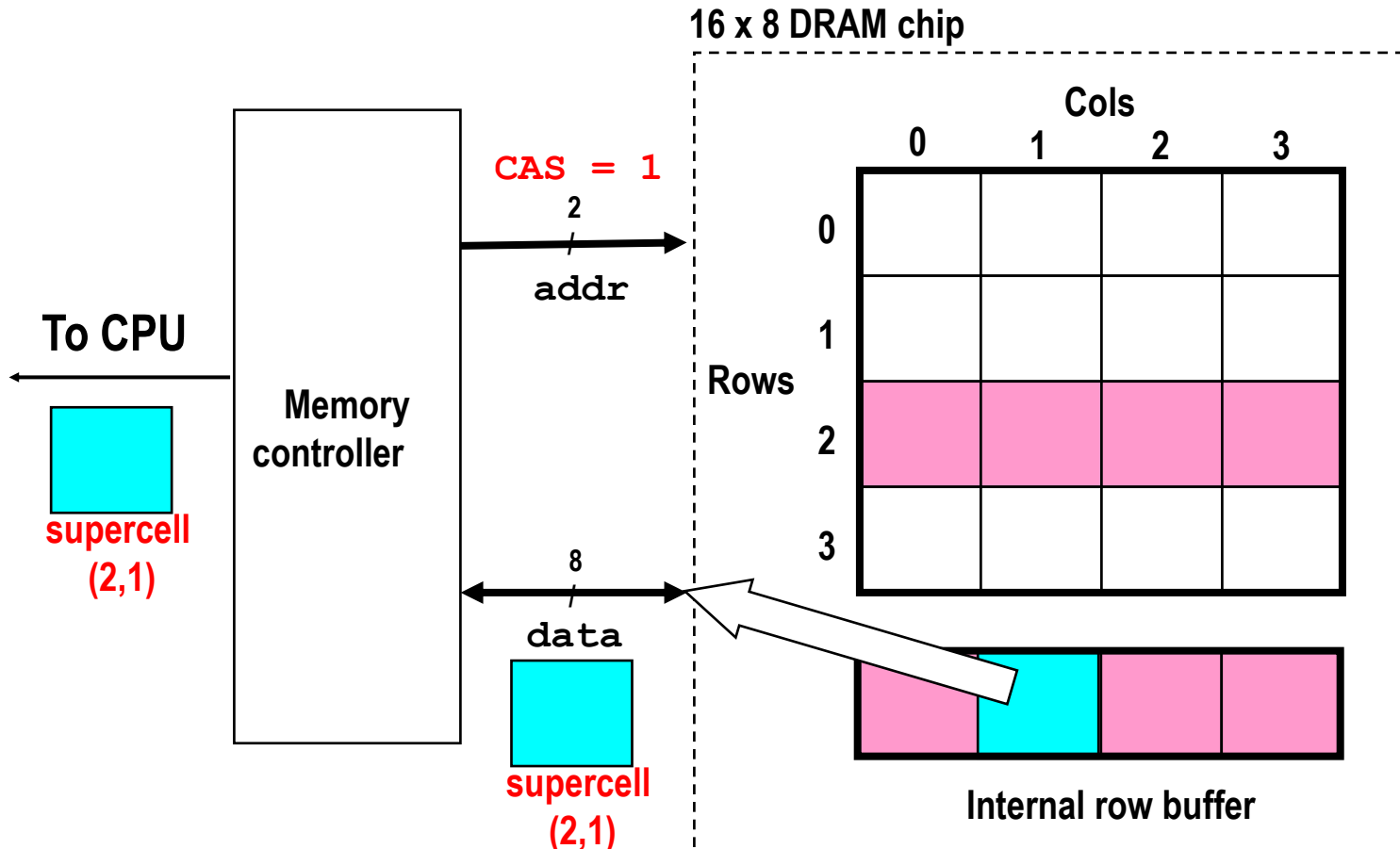
Step 1(b): Row 2 copied from DRAM array to row buffer.



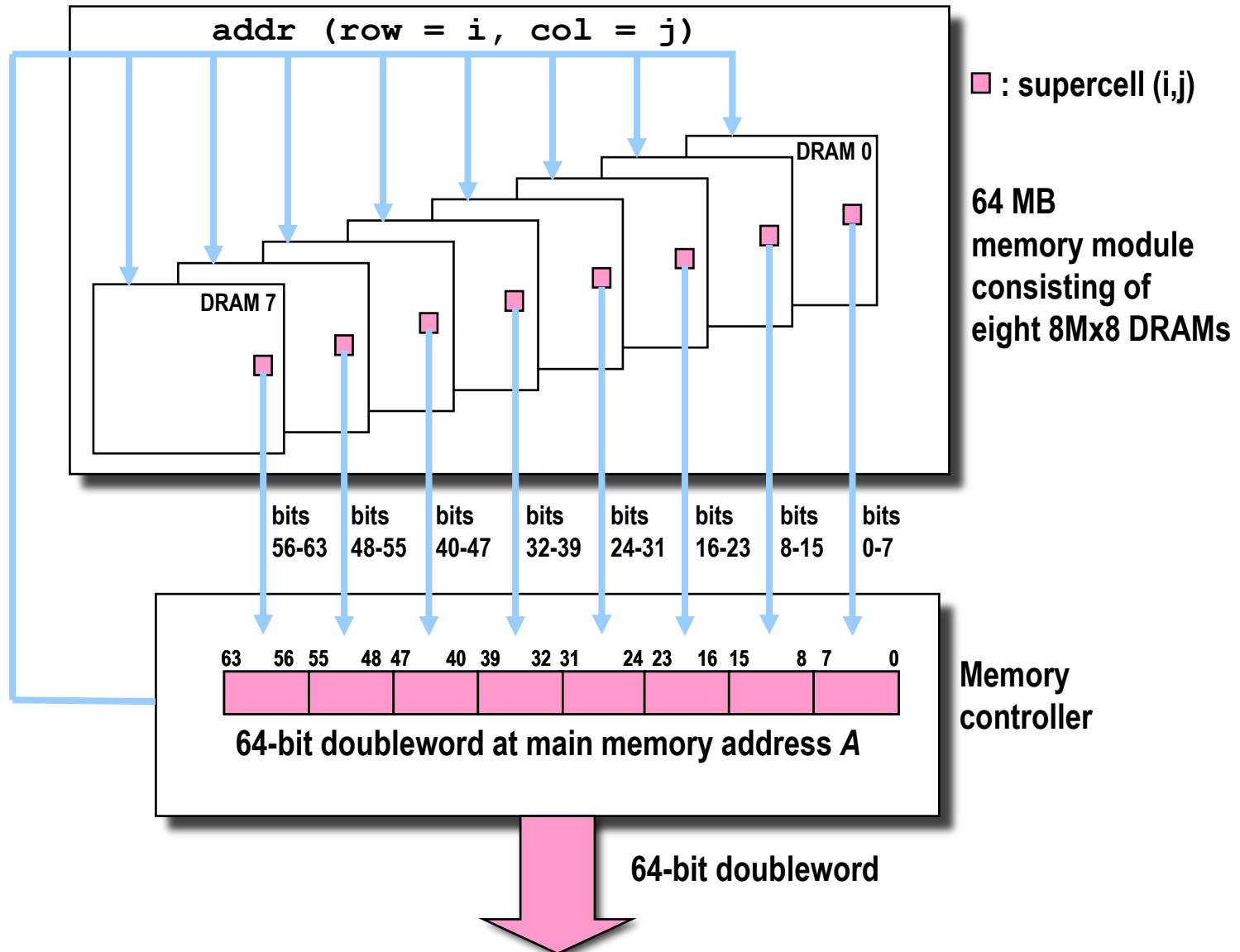
Reading DRAM Supercell (2,1)

Step 2(a): Column access strobe (**CAS**) selects column 1.

Step 2(b): Supercell (2,1) copied from buffer to data lines, and eventually back to the CPU.



Memory Modules



Aside: Nonvolatile Memories

- **DRAM and SRAM (caches) are volatile memories**
 - Lose information if powered off
- **Most common nonvolatile storage is the hard disk**
 - Rotating platters (like DVDs)... plentiful capacity, but very slow
- **Nonvolatile memories retain value even if powered off**
 - Read-only memory (**ROM**): programmed during production
 - Programmable ROM (**PROM**): can be programmed once
 - Erasable PROM (**EPROM**): can be bulk erased (UV, X-Ray)
 - Electrically erasable PROM (**EEPROM**): electronic erase capability
 - Flash memory: EEPROMs with partial (sector) erase capability
 - Wears out after about 100,000 erasings
- **Uses for Nonvolatile Memories**
 - Firmware programs stored in a ROM (BIOS, controllers for disks, network cards, graphics accelerators, security subsystems,...)
 - Solid state disks (replace rotating disks in thumb drives, smart phones, mp3 players, tablets, laptops,...)
 - Disk caches

Issue: memory access is slow

■ DRAM access is much slower than CPU cycle time

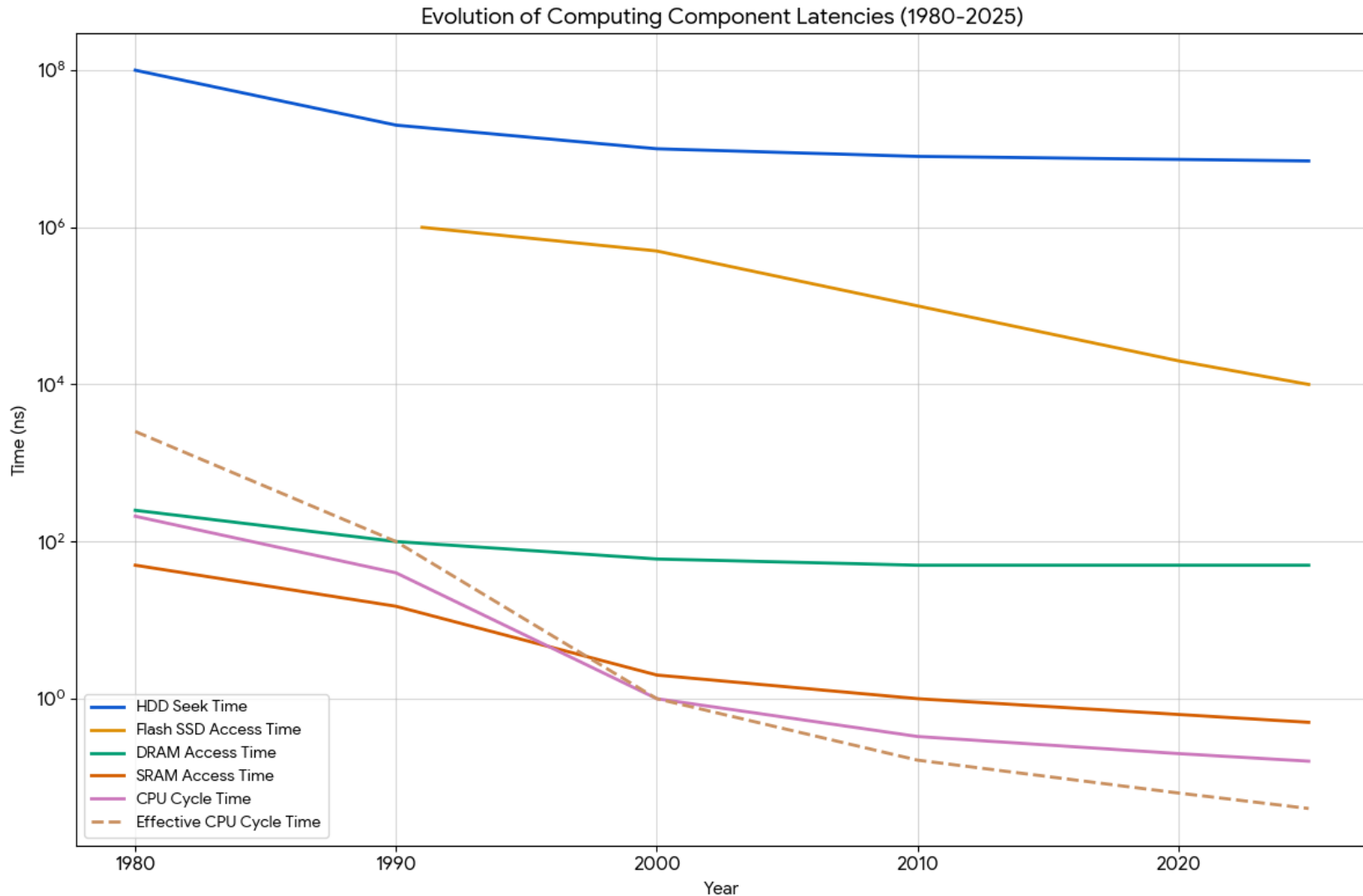
- A DRAM chip has access times of 30-50ns
 - and, transferring from main memory into register can take 3X or longer than that
- With sub-nanosecond cycles times, 100s of cycles per memory access
 - and the gap grows over time

■ Consequence: memory access efficiency crucial to performance

- approximately 1/3 of instructions are loads or stores
- both hardware and programmer have to work at it

The CPU-Memory Gap

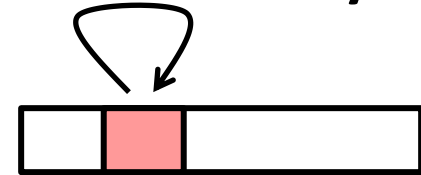
The gap widens between DRAM, disk, and CPU speeds.



Locality

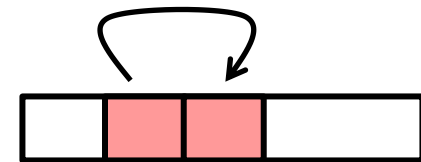
The key to bridging this CPU-Memory gap is a fundamental property of computer programs known as **locality**

- **Principle of Locality:** Programs tend to use data and instructions with addresses near or equal to those they have used recently



- **Temporal locality:**
 - Recently referenced items are likely to be referenced again in the near future

- **Spatial locality:**
 - Items with nearby addresses tend to be referenced close together in time



Locality Example

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```

■ Data references

- Reference array elements in succession (stride-1 reference pattern).
- Reference variable `sum` each iteration.

Spatial locality

Temporal locality

■ Instruction references

- Reference instructions in sequence.
- Cycle through loop repeatedly.

Spatial locality

Temporal locality

Qualitative Estimates of Locality

- **Claim:** Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer.
- **Question:** Does this function have good locality with respect to array *a*?

```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Locality Example

- **Question:** Does this function have good locality with respect to array *a*?

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Memory Hierarchies

- **Some fundamental and enduring properties of hardware and software:**
 - Fast storage technologies cost more per byte, have less capacity, and require more power (heat!)
 - The gap between CPU and main memory speed is widening
 - Well-written programs tend to exhibit good locality
- **These fundamental properties complement each other beautifully**
- **They suggest an approach for organizing memory and storage systems known as a **memory hierarchy****